

版本控制系统Git、合并冲突及其 消解研究

报告人：何 华

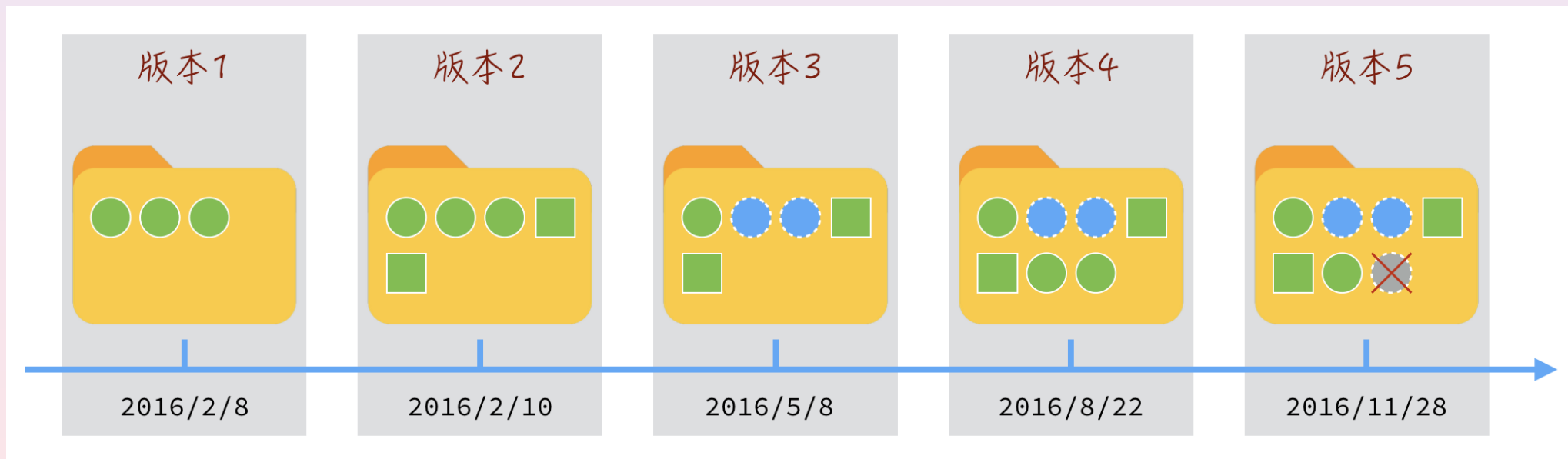


- ① 版本控制系统Git
- ② 代码合并和合并冲突
- ③ C/C++上合并冲突消解的一些研究
- ④ 总结

什么是版本控制？

- 绝大部分人无意使用过。
- 论文、slides、设计稿、游戏资源文件、软件的 v1, v2, v3, ..., v7
- 当某个较旧的资源在未来可能被重新用到的时候，就需要版本控制。

★ 其实我们需要的是台 “Time Machine” 。

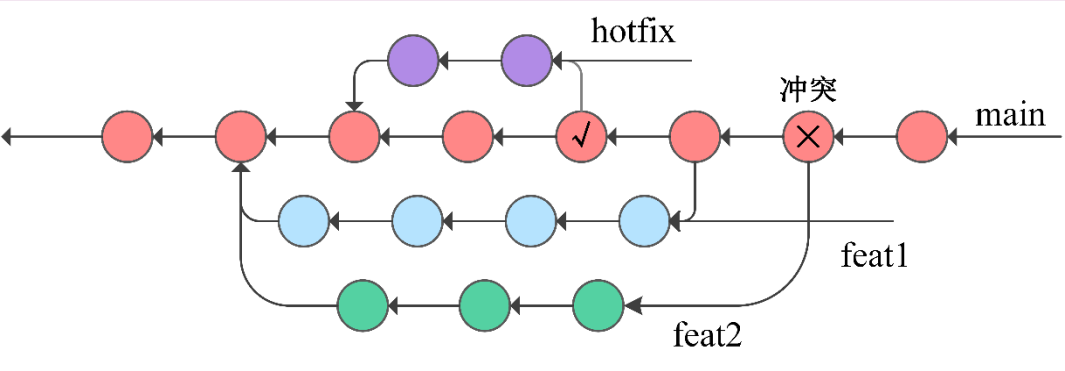


版本控制

- 将代码仓库的不同版本以链的形式组织起来，开发者可以在历史上回溯，分叉，修改。

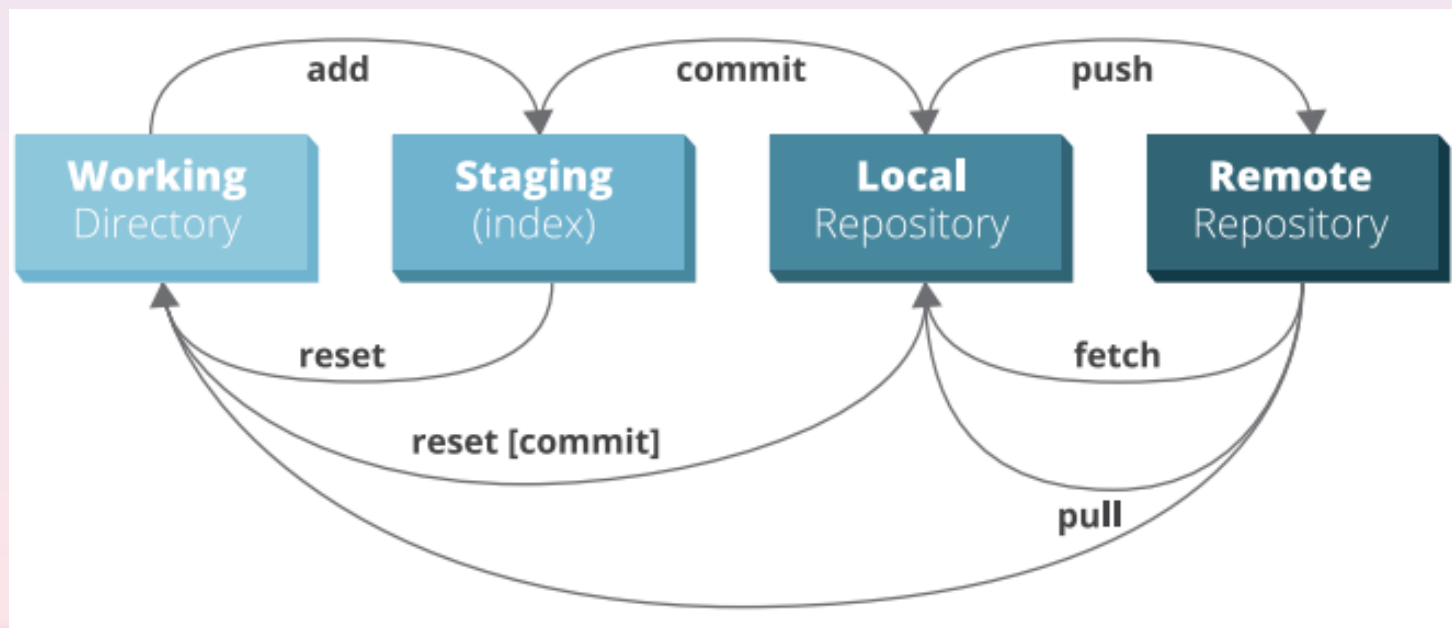
协作开发

- 高性能、去中心化、快照式存储和对多分支工作流的良好支持，允许开发者并发协作。



```
$ TZ=PST8PDT git log-compact --decorate --graph -n 17 v2.6.1
== 2015-09-28 ==
* 22f698cb 19:19 jch (tag: v2.6.1) Git 2.6.1
* 3adc4ec7 19:16 jch Sync with v2.5.4
\
* 24358560 15:34 jch (tag: v2.5.4) Git 2.5.4
* 11a458be 15:33 jch Sync with 2.4.10
\
* a2558fb8 15:30 jch (tag: v2.4.10) Git 2.4.10
* 6343e2f6 15:28 jch Sync with 2.3.10
\
* 18b58f70 15:26 jch (tag: v2.3.10, maint-2.3) Git 2.3.10
* 92cdfd21 14:59 jch Merge branch 'jk/xdiff-memory-limits' into maint-2.3
\
* 83c4d380 14:58 jk merge-file: enforce MAX_XDIFF_SIZE on incoming files
* dcd1742e 14:57 jk xdiff: reject files larger than ~1GB
* 3efb9880 14:57 jk react to errors in xdi_diff
* f2df3104 14:46 jch Merge branch 'jk/transfer-limit-redirection' into maint-2.3
\
== 2015-09-25 ==
* b2581164 15:32 bb http: limit redirection depth
* f4113cac 15:30 bb http: limit redirection to protocol-whitelist
* 5088d3b3 15:28 jk transport: refactor protocol whitelist code
\
== 2015-09-28 ==
* df37727a 14:33 jch Merge branch 'jk/transfer-limit-protocol' into maint-2.3
\
\
\
\
\
\
\
\
\
\
== 2015-09-23 ==
* 33cfcbb 11:35 jk submodule: allow only certain protocols for submodule fetches
```

- 分为工作区 (working directory)、暂存区 (staging area) 和对象数据库 (object database)。
 - 工作目录记录的是当前项目目录下的文件内容和状态，包含正在进行的编辑
 - 暂存区是持久化版本的前置区域，可以用来精细控制工作区中哪些修改会被提交到对象数据库中
 - 对象数据库 (.git 目录) 通过存储不同类别的二进制对象持久化了软件的不同版本
- 分别对应 项目根目录、.git/index, .git/objects 三个目录。



Git的版本控制原理（构成角度）

- 对象数据库通过存储不同类别的二进制对象持久化软件的不同版本。
- 每个对象会经由哈希函数（SHA1）生成一个160bit（40 hex digits）的索引。
- 不同的二进制对象被压缩后存储在 .git/objects目录下，取前两个hex digits划分目录加速查找

二进制对象一般结构

object tag	
' '	(single space)
size of object data	(in bytes)
'\0'	(null character)
object binary data	

BLOB（存储源代码和二进制文件）

"blob"	(blob object tag)
' '	(single space)
size of blob data	(in bytes)
'\0'	(null character)
blob data	

tree object（某个版本/staging area）

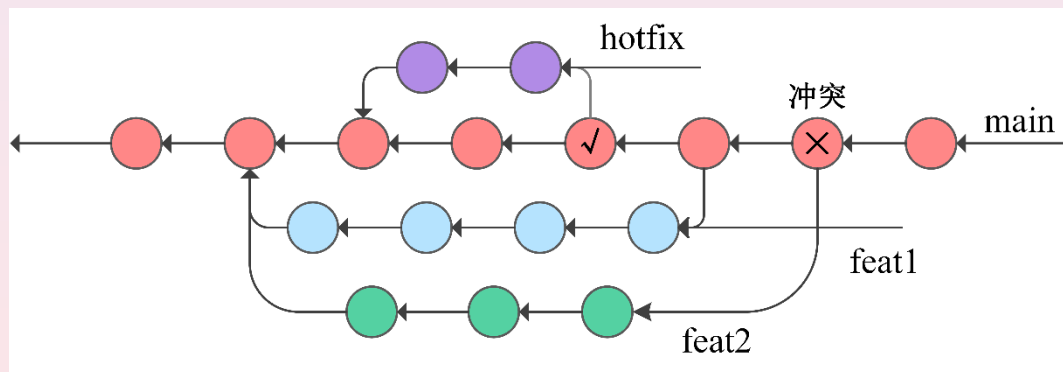
"tree"	(tree object tag)
' '	(single space)
size of tree data	(in bytes)
'\0'	(null character)
file 1 mode	(octal number)
' '	
file 1 path	
'\0'	
blob 1 SHA-1 hash	(hash of deflated blob object)
file 2 mode	
' '	
file 2 path	
'\0'	
blob 2 SHA-1 hash	
...	
file N mode	
' '	
file N path	
'\0'	
blob N SHA-1 hash	

commit object（某个版本）

"commit"	(commit object tag)
' '	(single space)
size of commit data	(in bytes)
'\0'	(null character)
tree SHA-1 hash	(hash value of committed tree)
parent 1 SHA-1 hash	(hash value of first parent tree)
parent 2 SHA-1 hash	(hash value of second parent tree)
...	
author ID email date	
committer ID email date	
	(empty line)
user comment	

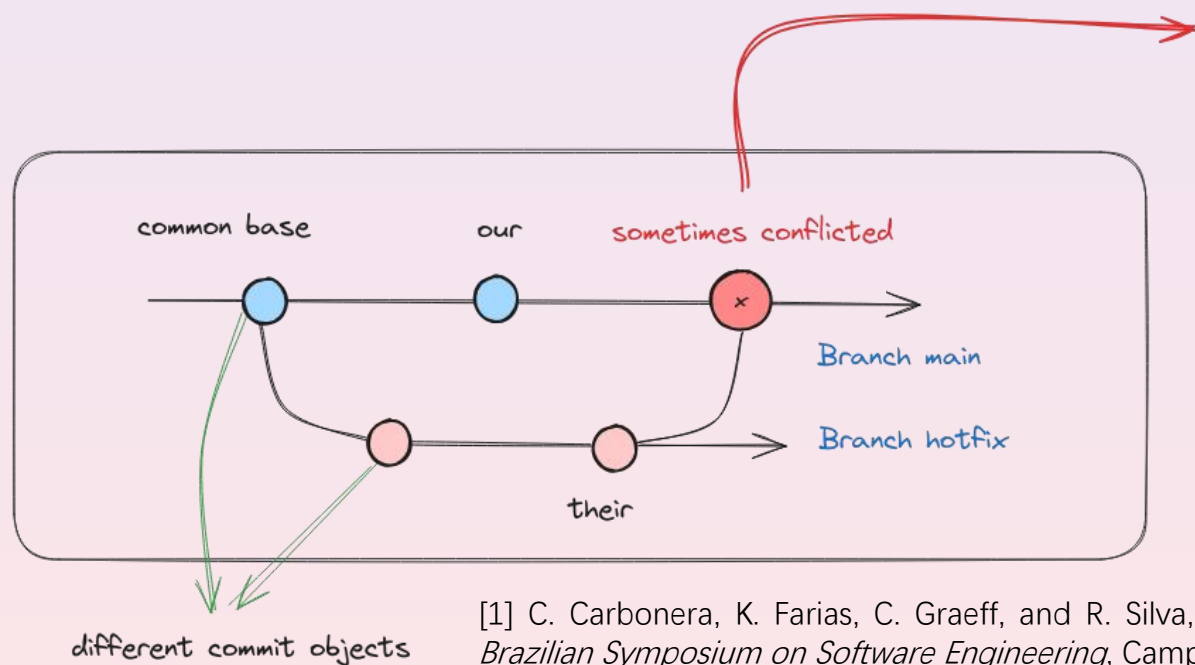
- 通过对象数据库，Git将软件不同的版本以链的形式组织起来。
- 在Git对象数据库上支持多分支 workflow 是容易的，开发者可以在多分支上协作编程。
 - commit object 内部成链
 - branch, tag, HEAD 记录的都是某个commit object的哈希标识。

```
(base) whalien@arch in .git/objects on eva » ls
00 08 10 19 22 2b 35 3e 48 51 5a 63 6b 75 7f 89 91 99 a1 a9 b1 bc c4 cd d5 de e7 f1 f9
01 09 11 1a 23 2d 37 3f 49 52 5b 64 6c 76 80 8a 92 9a a2 aa b3 bd c5 ce d6 df e9 f2 fa
02 0a 12 1b 24 2e 38 40 4b 54 5c 65 6d 78 81 8b 93 9b a3 ab b4 be c6 cf d7 e0 eb f3 fb
03 0b 13 1c 25 2f 39 41 4c 55 5d 66 6e 79 82 8c 94 9c a4 ac b6 bf c7 d0 d8 e1 ec f4 fc
04 0c 14 1d 26 30 3a 42 4d 56 5e 67 6f 7b 83 8d 95 9d a5 ad b7 c0 c8 d1 d9 e2 ed f5 fe
05 0d 15 1e 27 31 3b 43 4e 57 5f 68 71 7c 84 8e 96 9e a6 ae b8 c1 c9 d2 db e4 ee f6 ff
06 0e 16 1f 28 33 3c 46 4f 58 60 69 73 7d 85 8f 97 9f a7 af b9 c2 ca d3 dc e5 ef f7 info
07 0f 18 21 2a 34 3d 47 50 59 61 6a 74 7e 88 90 98 a0 a8 b0 ba c3 cb d4 dd e6 f0 f8 pack
```



代码合并和合并冲突

- 现代软件开发要求**协作编辑、随时提交（版本控制）**。去中心化的版本控制系统 Git 以支持多分支工作流、高效的特性得到了广泛应用。
- 不同分支上的开发者对代码的**同一区块做了不一致的修改**，代码集成的时候就会报告**合并冲突**。
- 合并冲突是**常见的[1]**。
- 合并冲突消解是**无聊、耗时的[1]**。



A conflict merge scenario

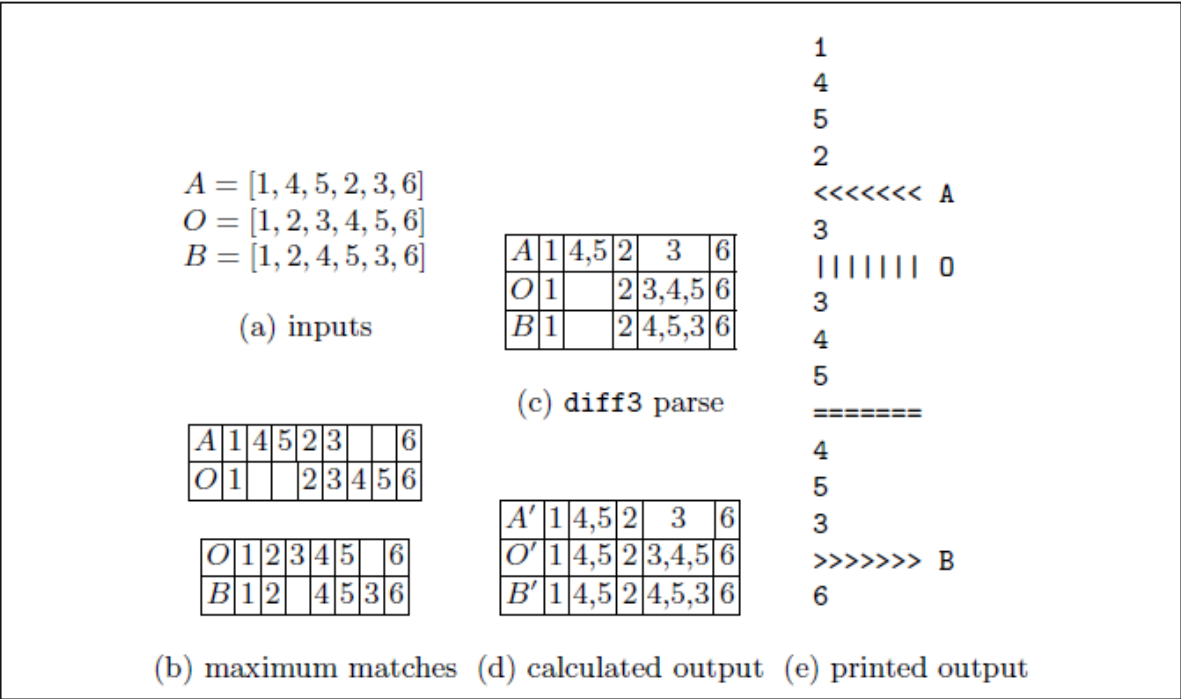
```
base.cc:125 { } rocksdb > EnvOptions()
base.cc:127 env_options->set_fd_cloexec = options.is_fd_close_on_exec;
base.cc:128 env_options->bytes_per_sync = options.bytes_per_sync;
base.cc:129 }
base.cc:130 }
base.cc:131 }
base.cc:132 EnvOptions::EnvOptions(const Options& options) {
base.cc:133 AssignEnvOptions(this, options);
base.cc:134 }
base.cc:135 }
base.cc:136 EnvOptions::EnvOptions() {
base.cc:137 Options options;
base.cc:138 AssignEnvOptions(this, options);
base.cc:139 }

our.cc:125 env_options->set_fd_cloexec = options.is_fd_close_on_exec;
our.cc:127 env_options->bytes_per_sync = options.bytes_per_sync;
our.cc:128 }
our.cc:129 }
our.cc:130 }
our.cc:131 }
our.cc:132 EnvOptions::EnvOptions(const DBOptions& options) {
our.cc:133 AssignEnvOptions(this, options);
our.cc:134 }
our.cc:135 }
our.cc:136 EnvOptions::EnvOptions() {
our.cc:137 DBOptions options;
our.cc:138 AssignEnvOptions(this, options);
our.cc:139 }

their.cc:236 }
their.cc:237 }
their.cc:238 }
their.cc:239 }
their.cc:240 }
their.cc:241 }
their.cc:242 }
their.cc:243 }
their.cc:244 }
their.cc:245 }
their.cc:246 }
their.cc:247 }
their.cc:248 }
their.cc:249 }
their.cc:250 }
their.cc:251 }
their.cc:252 }
their.cc:253 }
their.cc:254 }
their.cc:255 }
their.cc:256 }
their.cc:257 }
their.cc:258 }
their.cc:259 }
their.cc:260 }
their.cc:261 }
their.cc:262 }
their.cc:263 }
their.cc:264 }
their.cc:265 }
their.cc:266 }
their.cc:267 }
their.cc:268 }
their.cc:269 }
their.cc:270 }
their.cc:271 }
their.cc:272 }
their.cc:273 }
their.cc:274 }
their.cc:275 }
their.cc:276 }
their.cc:277 }
their.cc:278 }
their.cc:279 }
their.cc:280 }
their.cc:281 }
their.cc:282 }
their.cc:283 }
their.cc:284 }
their.cc:285 }
their.cc:286 }
their.cc:287 }
their.cc:288 }
their.cc:289 }
their.cc:290 }
their.cc:291 }
their.cc:292 }
their.cc:293 }
their.cc:294 }
their.cc:295 }
their.cc:296 }
their.cc:297 }
their.cc:298 }
their.cc:299 }
their.cc:300 }
their.cc:301 }
their.cc:302 }
their.cc:303 }
their.cc:304 }
their.cc:305 }
their.cc:306 }
their.cc:307 }
their.cc:308 }
their.cc:309 }
their.cc:310 }
their.cc:311 }
their.cc:312 }
their.cc:313 }
their.cc:314 }
their.cc:315 }
their.cc:316 }
their.cc:317 }
their.cc:318 }
their.cc:319 }
their.cc:320 }
their.cc:321 }
their.cc:322 }
their.cc:323 }
their.cc:324 }
their.cc:325 }
their.cc:326 }
their.cc:327 }
their.cc:328 }
their.cc:329 }
their.cc:330 }
their.cc:331 }
their.cc:332 }
their.cc:333 }
their.cc:334 }
their.cc:335 }
their.cc:336 }
their.cc:337 }
their.cc:338 }
their.cc:339 }
their.cc:340 }
their.cc:341 }
their.cc:342 }
their.cc:343 }
their.cc:344 }
their.cc:345 }
their.cc:346 }
their.cc:347 }
their.cc:348 }
their.cc:349 }
their.cc:350 }
their.cc:351 }
their.cc:352 }
their.cc:353 }
their.cc:354 }
their.cc:355 }
their.cc:356 }
their.cc:357 }
their.cc:358 }
their.cc:359 }
their.cc:360 }
their.cc:361 }
their.cc:362 }
their.cc:363 }
their.cc:364 }
their.cc:365 }
their.cc:366 }
their.cc:367 }
their.cc:368 }
their.cc:369 }
their.cc:370 }
their.cc:371 }
their.cc:372 }
their.cc:373 }
their.cc:374 }
their.cc:375 }
their.cc:376 }
their.cc:377 }
their.cc:378 }
their.cc:379 }
their.cc:380 }
their.cc:381 }
their.cc:382 }
their.cc:383 }
their.cc:384 }
their.cc:385 }
their.cc:386 }
their.cc:387 }
their.cc:388 }
their.cc:389 }
their.cc:390 }
their.cc:391 }
their.cc:392 }
their.cc:393 }
their.cc:394 }
their.cc:395 }
their.cc:396 }
their.cc:397 }
their.cc:398 }
their.cc:399 }
their.cc:400 }
```

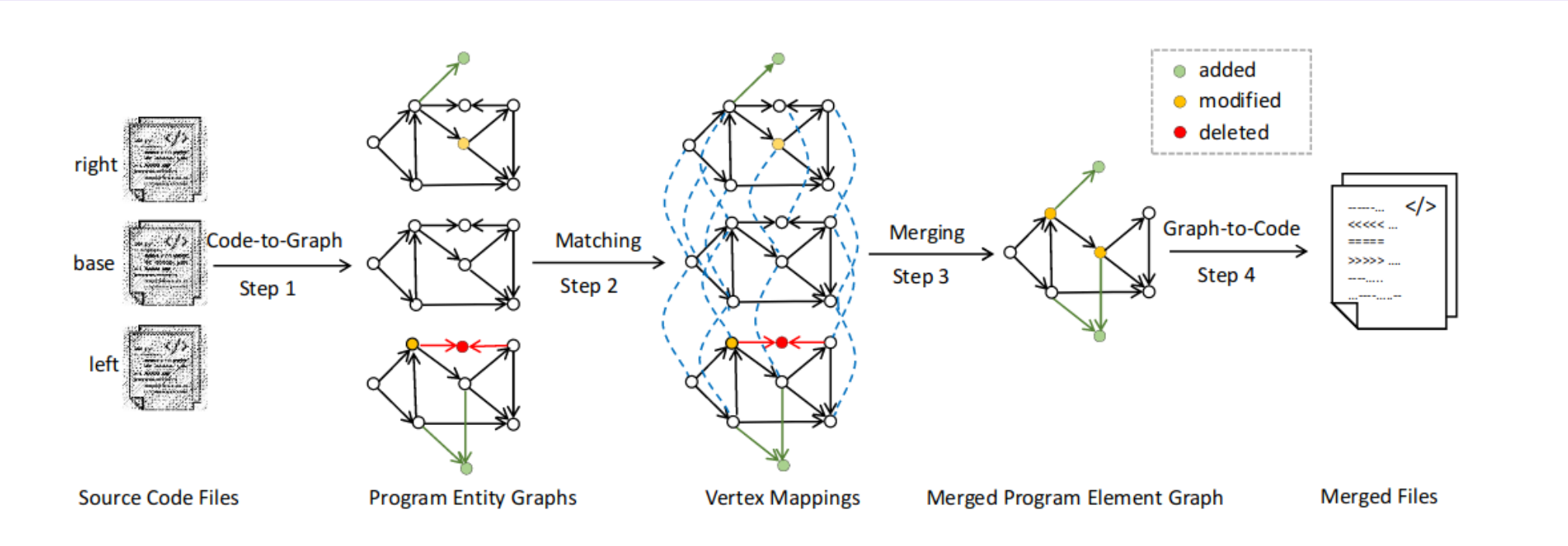
[1] C. Carbonera, K. Farias, C. Graeff, and R. Silva, "Software Merge: A Two-Decade Systematic Mapping Study," in *Proceedings of the XXXVII Brazilian Symposium on Software Engineering*, Campo Grande Brazil: ACM, Sep. 2023, pp. 99–108. doi: [10.1145/3613372.3613391](https://doi.org/10.1145/3613372.3613391).

基于程序结构的方法 - 非结构化合并



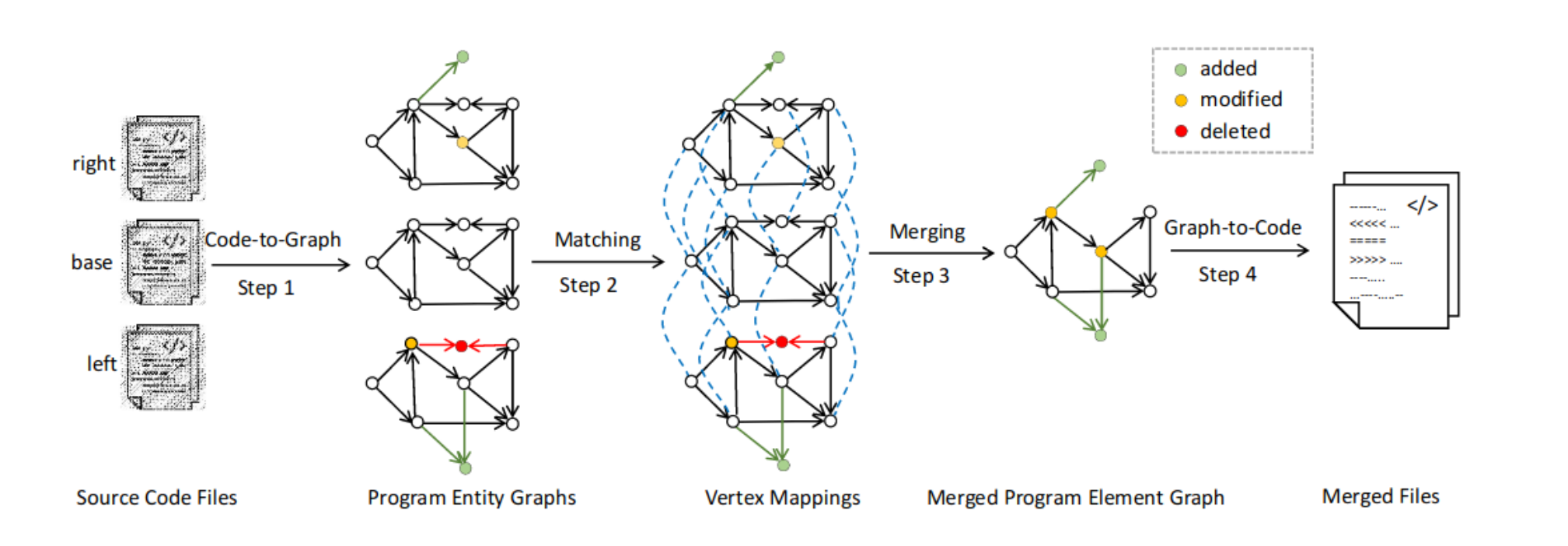
- 1. 输入：三个版本代码 A、O、B
- 2. 根据文本行做 AO 和 BO 最长公共子序列匹配
- 3. 根据2的结果对齐三个序列，计算输出
- 4. 输出至文件

基于程序结构的方法 - （半）结构化合并



1. 输入：三个版本代码 A、O、B
2. 根据程序实体做AO和OB最长公共子序列匹配
3. 根据2的结果对齐三个序列，计算输出
4. 输出至文件

基于程序结构的方法 – （半）结构化合并



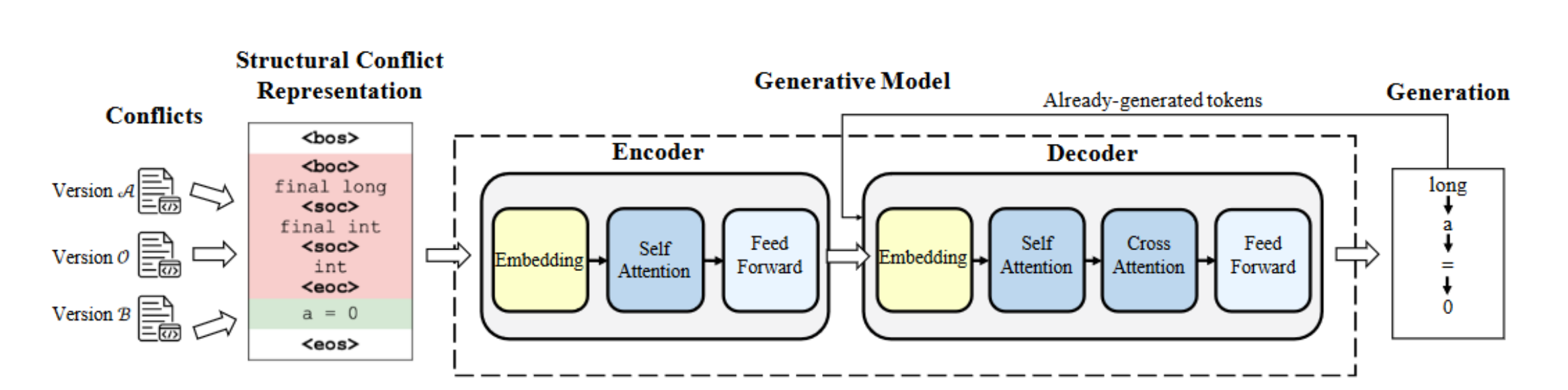
非结构化合并 – 树/图中结点为行

半结构化合并 – 树/图中结点为方法

结构化合并 – 树/图中结点为表达式等

三种方法区别在于抽象粒度

基于学习的方法 - 分类或生成



1. 将冲突经过某种结构化表示

2. 输入给经过微调的预训练模型

3. token 级别生成或分类

基于程序合成的方法

```
Forked Branch:
#include "chrome/browser/ui/views/accessibility/
  hc_with_theme_bubble_view.h"
Main Branch:
#include "chrome/browser/ui/views/accessibility/
  caption_bubble_controller_views.h"
```

```
Output r := Apply(c, t)
Condition c := And(p, c) | p
Predicate p := DuplicateMainFork(x)
              | DuplicateMainOutside(x)
              | DuplicateForkOutside(x)
              | MainSpecific(x) | ForkSpecific(x)
              | Dependency(x) | Rename(x)
              | FrequentPattern(x, path)
Transformation t := Concat(t, t) | Remove(s, s) | s
Selection s := Main(x) | Fork(x)
              | ForkByPath(x, path) | ForkByIndex(x, k)
              | MainByIndex(x, k) | MainByPath(x, path)
              | Pattern(x, key)
```

1. 通过大量的实证研究观察 合并冲突消解中的模式
2. 通过一套DSL定义合并冲突消解的策略
3. 通过DSL驱动程序合成程序自动消解冲突

本质上是基于规则的消解

类别	关键技术	相关工作	特点
基于程序结构的方法	非结构化合并、文本行比对合并	Diff3、KDiff3	Baseline，只能检测不能自动消解、基本无错误合并
	结构化合并、细粒度程序结点合并	JDime、AutoMerge	冲突块减少 61%、耗时长、泛化性差
	半结构化合并、树/图合并	jFSTMerge、IntelliMerge	冲突块减少 58.90%、行准确率 88.48%；耗时较短、泛化性一般
基于学习的方法	token 分类模型、预训练模型微调	MergeBert	准确率 63.2%、耗时短、泛化性良好
	token 生成模型、预训练模型微调	MergeGen	准确率 69.2%、解决率 67.7% ；耗时短、泛化性良好、解决合并冲突时可以引入新 token
历史挖掘、重构发现	基于操作的方法、重构发掘、重构合并、操作重放	RefMerge	减少冲突块 21%、耗时较短、泛化性差、新颖
程序合成	基于规则的自动化合并	Microsoft Prose	准确率 93.2% ，耗时短，泛化性好，解决率 11.4%

- 基于规则的方法冲突**解决率低**、基于学习的方法解决**精确率一般**。基于程序结构的方法**实现复杂，耗时久**。
- 系统级编程语言 C/C++ 在复杂系统中广泛存在，但由于**程序结构复杂、语法上下文敏感、文本宏对 EBNF 的干扰**，数据集缺失等问题，相关研究很少。

➡ 我们希望探索 **半结构化合并算法** 和 **基于学习的方法** 在 C/C++ 上的可用性及效果。

- 程序结构复杂
- 宏对程序结构不可预测
- 无事实上的包管理器和清晰的路径划分，静态分析很困难
- 工具链不齐全
- 学界广泛使用的预训练模型在 C++ 上训练不足[1]
- 缺乏 C/C++ 相关合并冲突数据集[2]

[1] <https://huggingface.co/Salesforce/codet5-base>

[2] <https://zenodo.org/records/6366908>

- 以函数定义为例

```
// default definition
Identity(const Identity &I) = default;

// delete definition
Identity(const Identity &I) = delete;

Identity(int n) : rank(n) {} // init list

operator bool() const {
    // type cast operator
}

auto determinant() const -> double {
    // trailing return type
}

Matrix operator+(const Matrix& other) const {
    // normal function
}
```

```
// macro function
#define IDENTITY_MATRIX(size) ([](int s) { \
    Matrix m(s, s); \
    for(int i = 0; i < s; ++i) \
        m.data[i][i] = 1.0; \
    return m; \
})(size)

template<typename T>
typename std::enable_if<std::is_arithmetic<T>::value,
Matrix>::type
createIdentityMatrix(T size) {
    // template definition, SFINAE
}

template<typename T>
concept Arithmetic = std::is_arithmetic_v<T>;

template<Arithmetic T>
Matrix createIdentityMatrix(T size) {
    // template definition, C++20 concept
}
```

```
<class body> ::= { <class body declarations>? }
```

```
<class body declarations> ::= <class body declaration> | <class body declarations> <class body declaration>
```

```
<class body declaration> ::= <class member declaration> | <static initializer> | <constructor declaration>
```

```
<class member declaration> ::= <field declaration> | <method declaration>
```

- 宏可能为空
 - 理论上可以放在 EBNF 右侧的每一个位置，路径爆炸
- 宏可能替换成一个任意复杂的结构
 - 很难用上下文无关语法解析

```
#include <iostream>

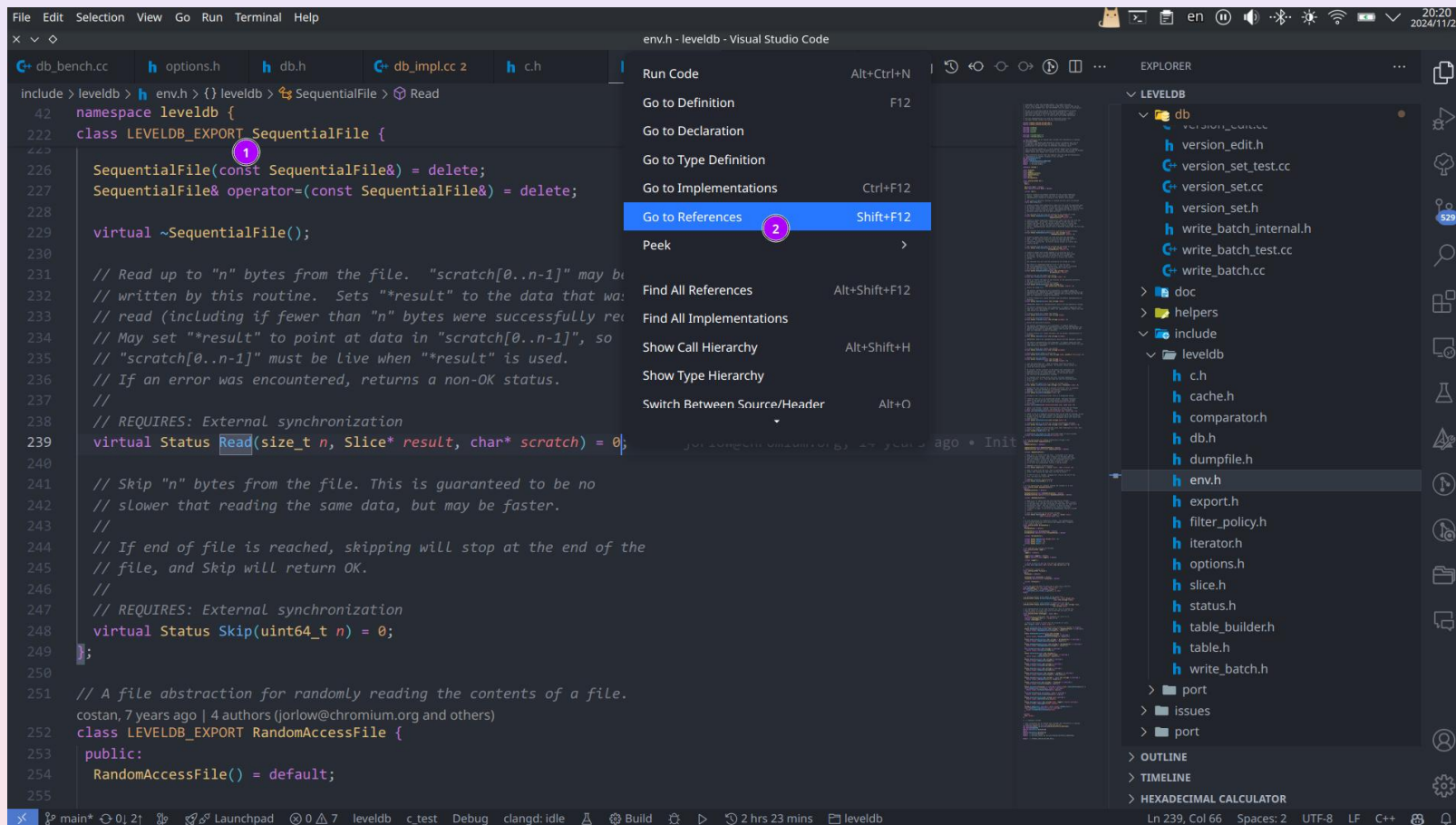
using namespace std;

auto main() -> int {
    cout << "Hello World!\n";
}
```

- 头文件、实现文件分离 (.h, .cc)
 - 静态分析困难, 即使是 clang, 查找对应的头/实现最坏可能fallback到启发式规则
- 无事实上的包管理标准, 查找依赖困难
 - 包管理: conan、vcpkg、系统自带、源码构建
 - 构建工具 make、configure、autoconfigure、cmake、sln、ninja、meson、xmake...
- 基于规则的工作出发点是 C/C++ 上工具链不齐全[1]

[1] R. Pan, V. Le, N. Nagappan, S. Gulwani, S. Lahiri, and M. Kaufman, "Can Program Synthesis be Used to Learn Merge Conflict Resolutions? An Empirical Analysis," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, Madrid, ES: IEEE, May 2021, pp. 785–796. doi: [10.1109/ICSE43902.2021.00077](https://doi.org/10.1109/ICSE43902.2021.00077).

➡ 核心问题：构建树/图 + 程序静态分析 + 一定的通用性



- 程序结构复杂

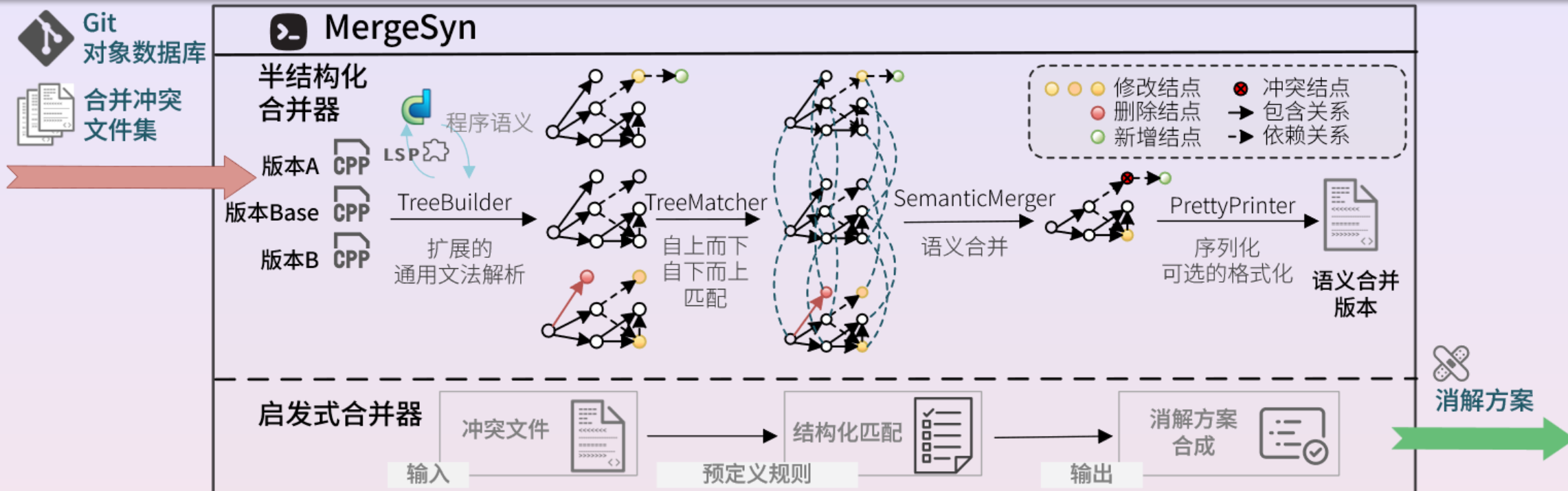
➡ 限定为 $\leq$ std c++17

- 程序实体树构建

➡ 编程语言着色技术
(多路径 GLR文法)

- 程序实体树匹配

➡ 经典的程序补全技术
(编程语言服务器协议)



- 基于现代代码编辑器技术构建半结构化合并算法。
 - TreeBuilder 基于 多路径 GLR、正则构建程序实体树；基于 LSP 抽取语义
 - TreeMatcher top-down、bottom-up 结合的 fuzzy matcher
 - SemanticMerger 程序实体语义感知的合并
- 吸收基于程序合成的合并高精确率的优点，实现启发式合并进行冲突消解效果补充。
 - 七条置信度比较高、易于实施的启发式规则

$$VT := \{TU, NS, CS, LS, FDEF, FO, FSM, TDECL, FDECL, OC, TN\}$$

- vertex type: 树结点类型

$$EC := \{contain, definition, declaration, inclusion, call\}$$

- edge correlation 边代表的关系

$$ET \in VT \times EC \times VT$$

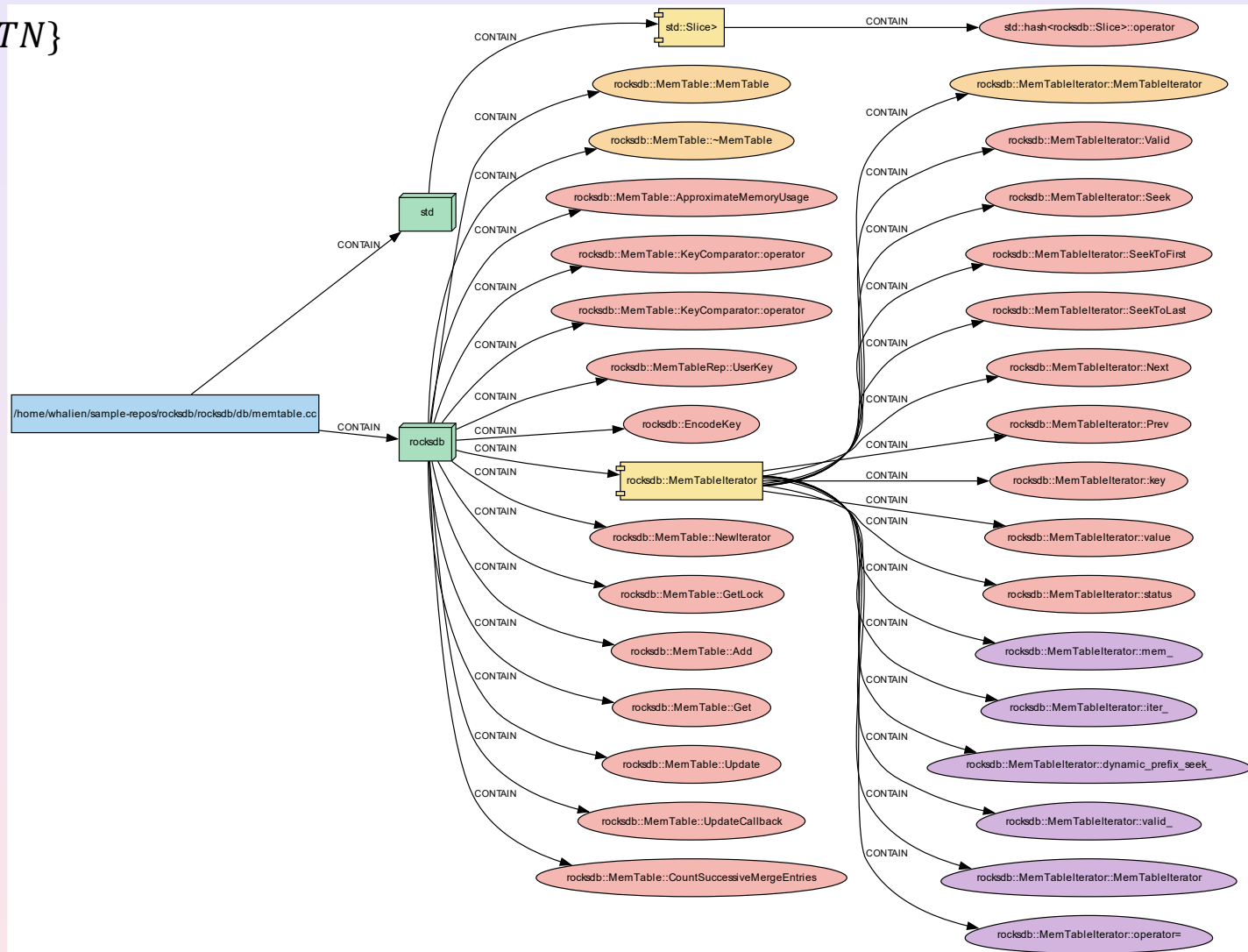
- edge type 边的类型

$$V \qquad \text{type} := (V \rightarrow VT)$$

- 顶点集合

$$Sig := V \rightarrow Signature$$

- 边集合 $ID := V \rightarrow R$



算法 1: 程序实体树的两路匹配算法

输入: Base 版本程序实体树 T_{base} , Revision 版本程序实体树 T_{revision}

输出: 匹配集合 M_{br} , 删除节点集合 U_{rd} , 新增节点集合 U_{ra}

```

1  $MAP_{\text{base}} \leftarrow \{ (ID(n), n) \mid n \in T_{\text{base}} \};$ 
2  $MAP_{\text{revision}} \leftarrow \{ (ID(n), n) \mid n \in T_{\text{revision}} \};$ 

3 // 自顶而下精准匹配
4  $M_{br} \leftarrow \emptyset, U_{rd} \leftarrow \emptyset, U_{ra} \leftarrow \emptyset;$ 

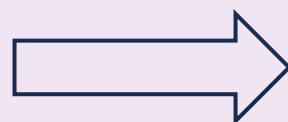
5 for 每个节点  $n \in T_{\text{base}}$  do
6   if  $ID(n) \in MAP_{\text{revision}}$  then
7     令  $n' \in T_{\text{revision}}$  满足  $ID(n) = ID(n')$ ;
8      $M_{br} \leftarrow M_{br} \cup \{ (n, n') \};$ 
9   else
10     $U_{rd} \leftarrow U_{rd} \cup \{ n \};$ 

11 for 每个节点  $n' \in T_{\text{revision}}$  do
12   if  $ID(n') \notin MAP_{\text{base}}$  then
13     $U_{ra} \leftarrow U_{ra} \cup \{ n' \};$ 

14 // 自底而上模糊匹配
15 for 每种节点类型  $v \in NT$  do
16    $U_{\text{base}}(v) \leftarrow \{ n \in U_{rd} \mid \text{Type}(n) = v \};$ 
17    $U_{\text{revision}}(v) \leftarrow \{ n' \in U_{ra} \mid \text{Type}(n') = v \};$ 
18   if  $U_{\text{base}}(v) \neq \emptyset$  且  $U_{\text{revision}}(v) \neq \emptyset$  then
19     定于节点类型  $v$  的相似度函数  $sim_v : U_{\text{base}}(v) \times U_{\text{revision}}(v) \rightarrow [0, 1];$ 
20     计算结点间的相似度矩阵
21      $Sim \leftarrow \{ (n, n', w) \mid w = sim_v(n, n'), w \geq \theta_v \};$ 
22     执行二分图匹配
23      $M_{sim} \leftarrow \text{bipartiteMatching}(U_{\text{base}}, U_{\text{revision}}, Sim, U_{rd}, U_{ra});$ 
24      $M_{br} \leftarrow M_{br} \cup M_{sim};$ 

25 return  $M_{br}, U_{rd}, U_{ra};$ 

```



基于结点类型、结点签名的严格匹配

旨在减少需 Fuzzy Match 的节点数目



假定重构后结点类型不变, 逐类型:

基于文本相似度 + 上下文相似度加权的二分图匹配

Algorithm 2: 语义合并算法

输入: M_{abb} : 三版本匹配集合
 U_{ra} : 待合并版本新增的转译单元集合
 U_{rd} : 待合并版本删除的转译单元集合
输出: TU_m : 语义合并后的程序实体树集合

```

1 Function threeWayMerge( $M_{abb}$ ,  $U_{ra}$ ,  $U_{rd}$ ):
2    $TU_m \leftarrow \emptyset$ ;
3    $N_{merge} \leftarrow \{(n_a, n_{base}, n_b) \mid n_{base} \in M_{abb} \wedge n_{base} \notin U_{rd}\}$ ;
4   for 每个三元组  $(n_a, n_{base}, n_b) \in N_{merge}$  do
5     if  $Type(n_{base}) = TU$  then
6        $n_m \leftarrow mergeComposite(n_a, n_{base}, n_b)$ ;
7        $TU_m \leftarrow TU_m \cup \{n_m\}$ ;
8   for 每个  $n \in U_{ra}$  do
9      $TU_m \leftarrow TU_m \cup \{n\}$ ;
10  return  $TU_m$ ;

11 Function mergeComposite( $n_a, n_{base}, n_b$ ):
12   $n_m \leftarrow \phi$ ;
13   $n_m \leftarrow mergeComment(n_a, n_{base}, n_b)$ ;
14   $n_m \leftarrow mergeSignature(n_a, n_{base}, n_b)$ ;
15   $C_n \leftarrow \{(c_a, c_{base}, c_b) \mid c_{base} \in Children(n_{base}) \wedge (c_a, c_{base}, c_b) \in M_{abb}\}$ ;
16  for 每个三元组  $(c_a, c_{base}, c_b) \in C_n$  do
17    if  $isTerminal(c_{base})$  then
18       $c_m \leftarrow mergeTerminal(c_a, c_{base}, c_b)$ ;
19    else
20       $c_m \leftarrow mergeComposite(c_a, c_{base}, c_b)$ ;
21     $n_m.appendChild(c_m)$ ;
22  return  $n_m$ ;

23 Function mergeTerminal( $c_a, c_{base}, c_b$ ):
24   $c_m \leftarrow \phi$ ;
25   $c_m \leftarrow mergeComment(c_a, c_{base}, c_b)$ ;
26   $c_m \leftarrow mergeSignature(c_a, c_{base}, c_b)$ ;
27   $c_m \leftarrow mergeComponents(c_a, c_{base}, c_b)$ ;
28  return  $c_m$ ;
    
```

- 以需要合并的 Translation Unit 为单位 进行 DFS 合并

- Composite Node

- 注释、签名、孩子结点

- $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

$A \rightarrow F \rightarrow C \rightarrow G \rightarrow D \rightarrow E$

拓扑排序、图合并

- Terminal Node

- 注释、签名、attribute、函数参数

- 调用 GitMerge 做文本合并

七条置信度比较高、易于实施的启发式规则：

(1) 预处理等价规则

$$\frac{\text{Identical}(C_A, C_B)}{\text{resolution} \leftarrow C_A}$$

(2) 事实单边修改规则

$$\frac{(\text{Identical}(C_A, C_O) \vee \text{Identical}(C_B, C_O)) \wedge \neg \text{Identical}(C_A, C_B)}{\text{resolution} \leftarrow \begin{cases} C_B & \text{if Identical}(C_A, C_O) \\ C_A & \text{otherwise} \end{cases}}$$

(3) 包含关系优化规则

$$\frac{\text{Subset}(C_A, C_B) \vee \text{Subset}(C_B, C_A)}{\text{resolution} \leftarrow \begin{cases} C_B & \text{if Subset}(C_A, C_B) \\ C_A & \text{otherwise} \end{cases}}$$

...

```
#include <sys/time.h>
#include <sys/uio.h>

#include <limits.h>
#include <stdarg.h>
#include <stdio.h>
#include <termios.h>
#include <wchar.h>

struct args;
struct args_command_state;
struct client;
struct cmd;
struct cmd_find_state;
struct cmdq_item;
struct cmdq_list;
```

```
<<<<<< HEAD
|||||||
void checkout_revision(const std::string &db_path,
const std::string &revision) {
    // ...
}
=====
void checkout_revision(std::string_view db_path,
std::string_view revision) {
    // ...
}
>>>>>> ac868c
```

- 数据集

- 来自AOSP和Github C/C++ Hot 100 的44个C/C++项目, 7213个合并冲突场景, 14873个冲突文件, 31597个冲突区块。
- 选取标准: fork数目, star数目, C/C++代码行数, 冲突数目。
- 数据集收集: 参考过往工作的思路, 编写脚本, 重放项目历史中的合并。
- 2941个冲突区块用于测试, 其余的用来微调对比验证

- 对比算法

- Baseline: GitMerge、无冲突解决能力, 只有冲突检测能力
- SOTA learning-based: 在huggingface 720k 个C/C++文件集[1]上微调 MergeGen 的 tokenizer。在自行收集的31597个冲突C/C++文件集上微调 CodeT5。

RQ1: (Overall Effectiveness) MergeSyn在冲突区块级别C/C++合并冲突消解的性能表现如何?

RQ2: (Ablation Study) MergeSyn中加入的启发式合并器能否很好的补充其半结构化合并器?

RQ3: (Efficiency Comparison) 不同技术路线的方法在消解合并冲突时的运行效率分布是怎样的?

RQ4: (Discussion) 不同技术路线的方法消解合并冲突时性能表现会受哪些因素影响?

表 3.2 MergeSyn 和 MergeGen 在待测项目上的性能表现

项目	MergeSyn		MergeGen	
	Precision	Accuracy	Precision	Accuracy
grpc	54.55%	34.84%	45.10%	44.52%
ClickHouse	36.73%	27.27%	32.81%	31.82%
rocksdb	59.77%	40.94%	30.58%	29.13%
mongo	62.50%	24.69%	28.75%	28.40%
rethinkdb	39.53%	22.52%	47.76%	42.38%

表 3.2 续 MergeSyn 和 MergeGen 在待测项目上的性能表现

项目	MergeSyn		MergeGen	
	Precision	Accuracy	Precision	Accuracy
frameworks/av	45.55%	36.25%	63.44%	60.00%
frameworks/native	44.68%	34.43%	42.37%	40.98%
hardware/interfaces	65.17%	52.73%	37.61%	37.27%
system/chre	55.36%	37.80%	48.75%	47.56%
system/core	53.66%	35.48%	51.67%	50.00%
system/extras	45.61%	37.14%	40.30%	38.57%
system/update/engine	56.19%	44.70%	55.47%	53.79%
git	73.08%	25.33%	34.25%	33.33%
lvgl	64.25%	43.66%	37.55%	35.56%
redis	70.00%	29.17%	33.33%	31.25%
sway	42.22%	29.69%	24.19%	23.44%
tmux	55.56%	39.22%	43.48%	39.22%
zstd	63.16%	36.00%	29.55%	26.00%
平均值	56.31%	35.65%	39.18%	37.52%

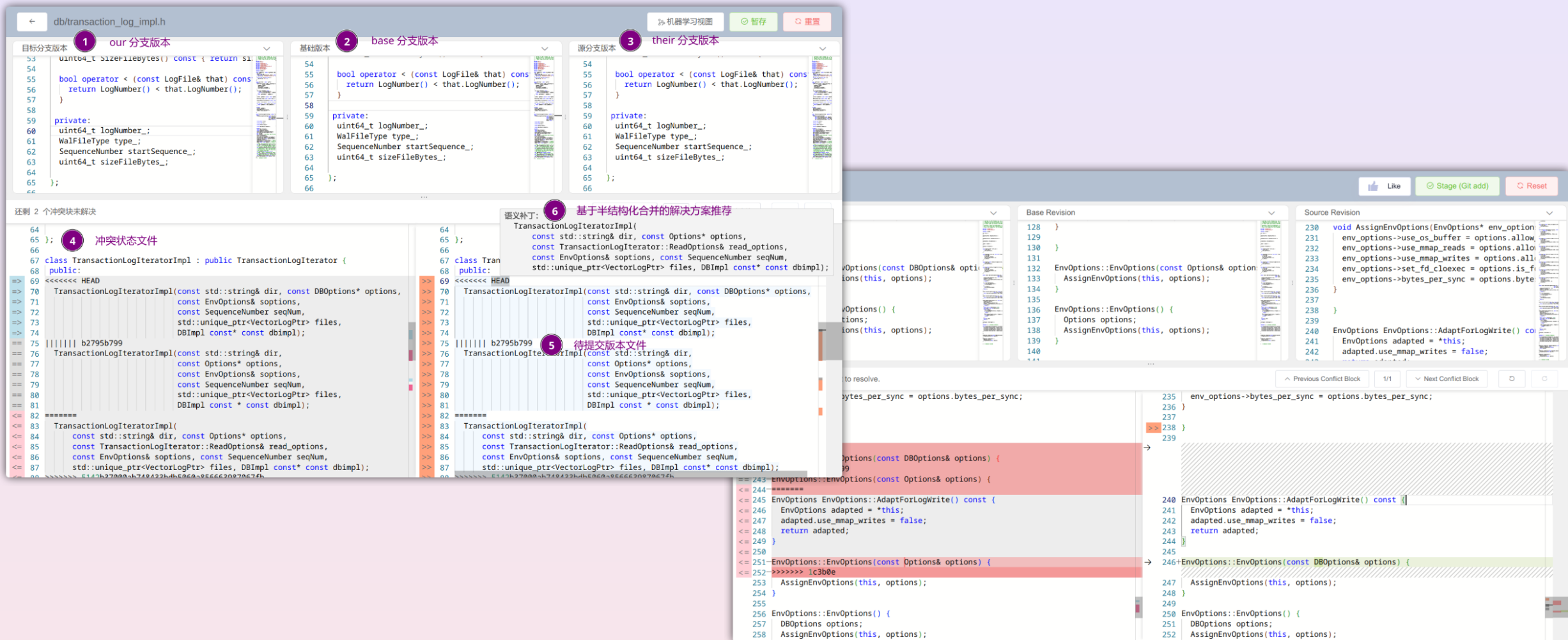
$Precision = \frac{Correct}{Given_{alg}}$ 算法给出解决方案的准确率

$Accuracy = \frac{Correct}{Total}$ 正确解决的块占所有块比例

★ 在冲突消解问题上，Precision 更重要[1]，因为开发者可以以更高的概率接受。

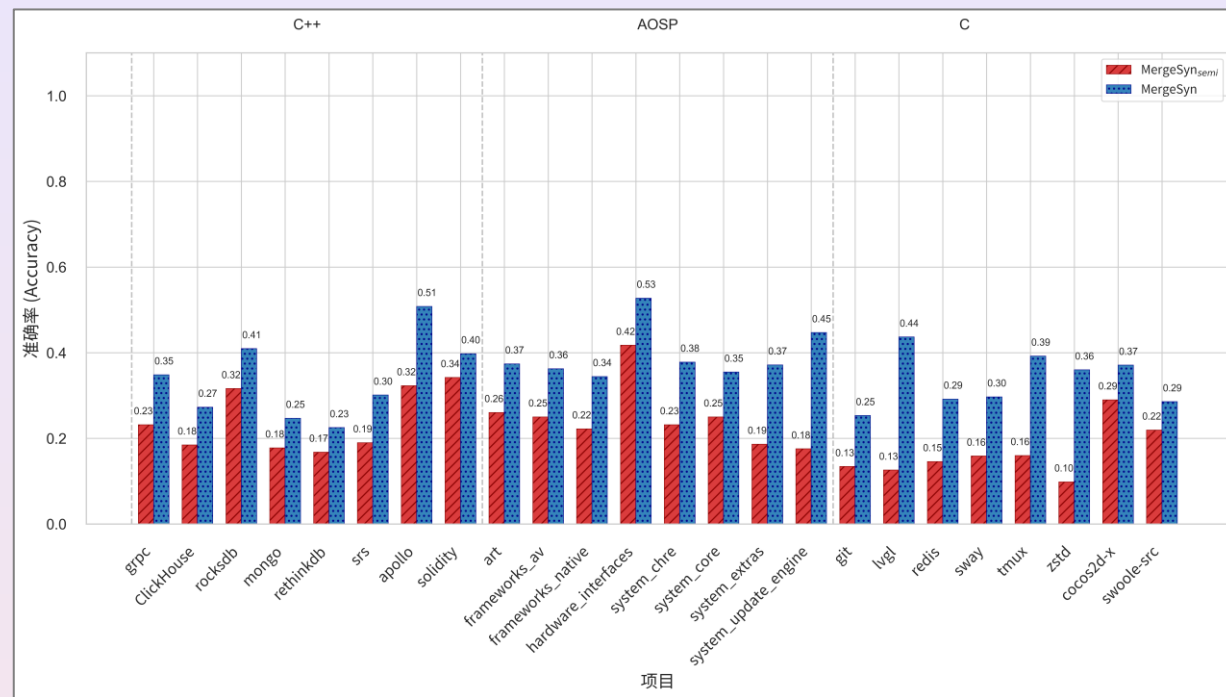
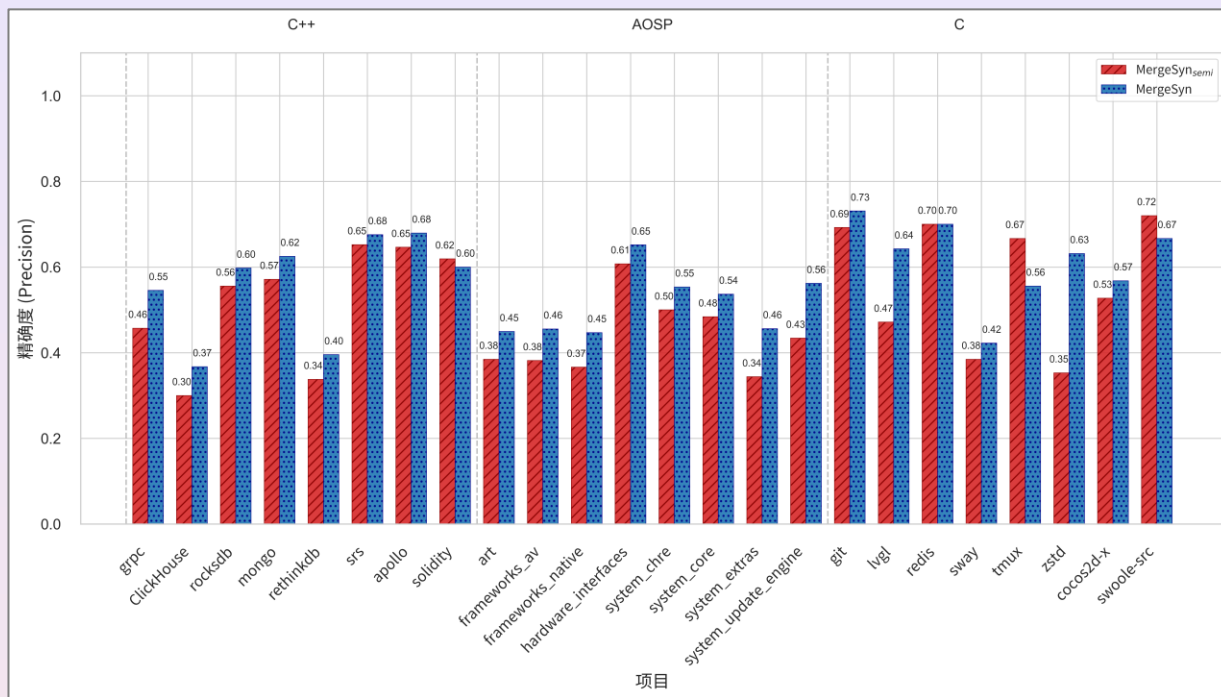
★ 后续工作？基于学习的高 Precision 方案

在C/C++上，MergeBot 有着比 MergeGen 更可靠的解决方案精确率 56.3% (39.2%)，仅降低 1.87%的准确率。
MergeGen在C/C++上出现了分布外泛化问题。



- 工具层面支持应用推荐消解方案后的字符级实时差异显示，开发者需要检查应用的冲突消解方案。

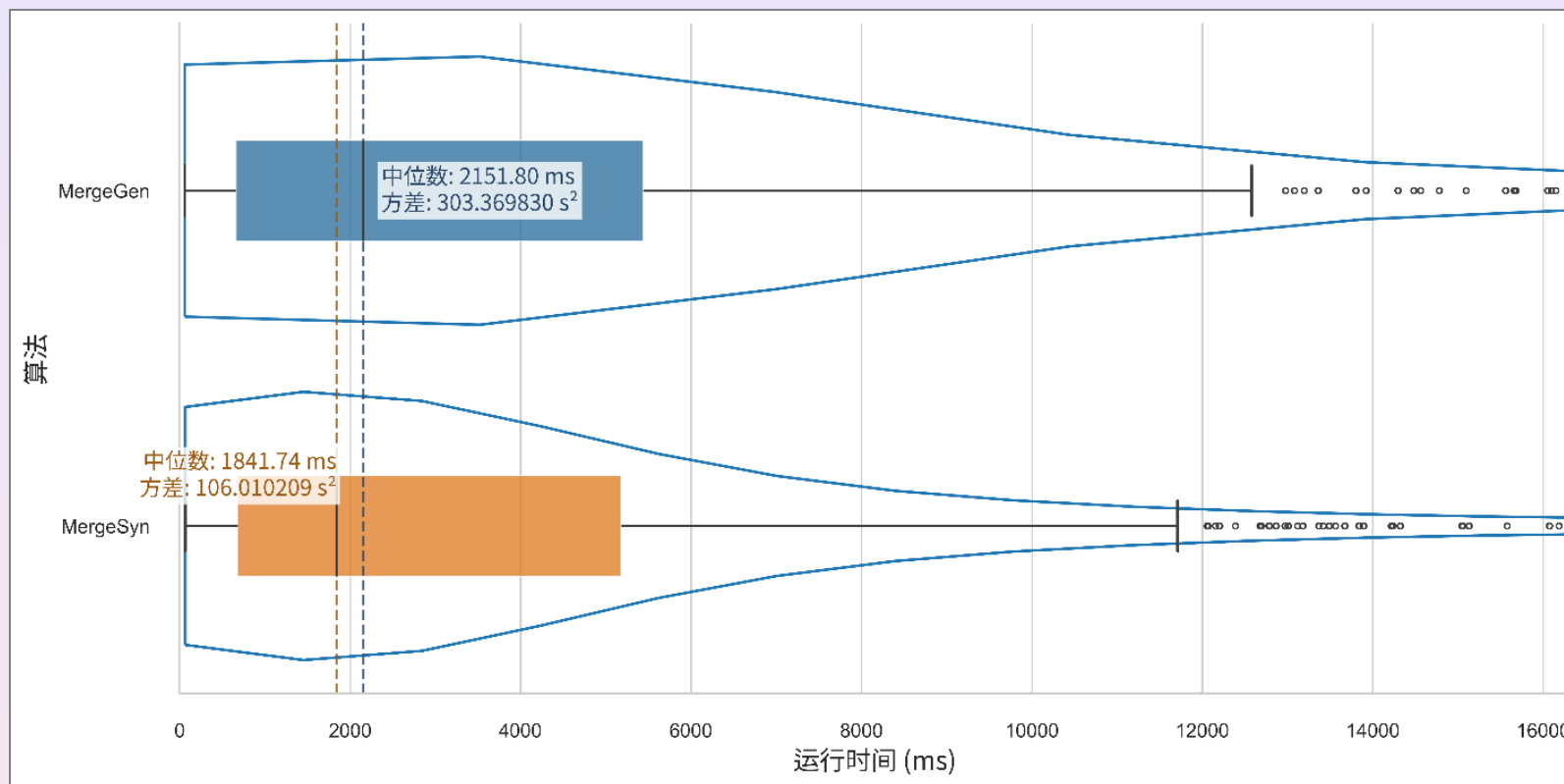
RQ2: Ablation Study



在被测项目上，MergeSyn的启发式合并器能够正向补充其半结构化合并器，平均精确率提升为5.79%；平均准确率提升为13.78%。

含启发式合并的MergeSyn在22/24个项目上表现出了更优的Precision，在24/24个项目上都表现出了更优的Accuracy，说明确定性策略在消解合并冲突时有一定优势。

RQ3: Efficiency Comparison



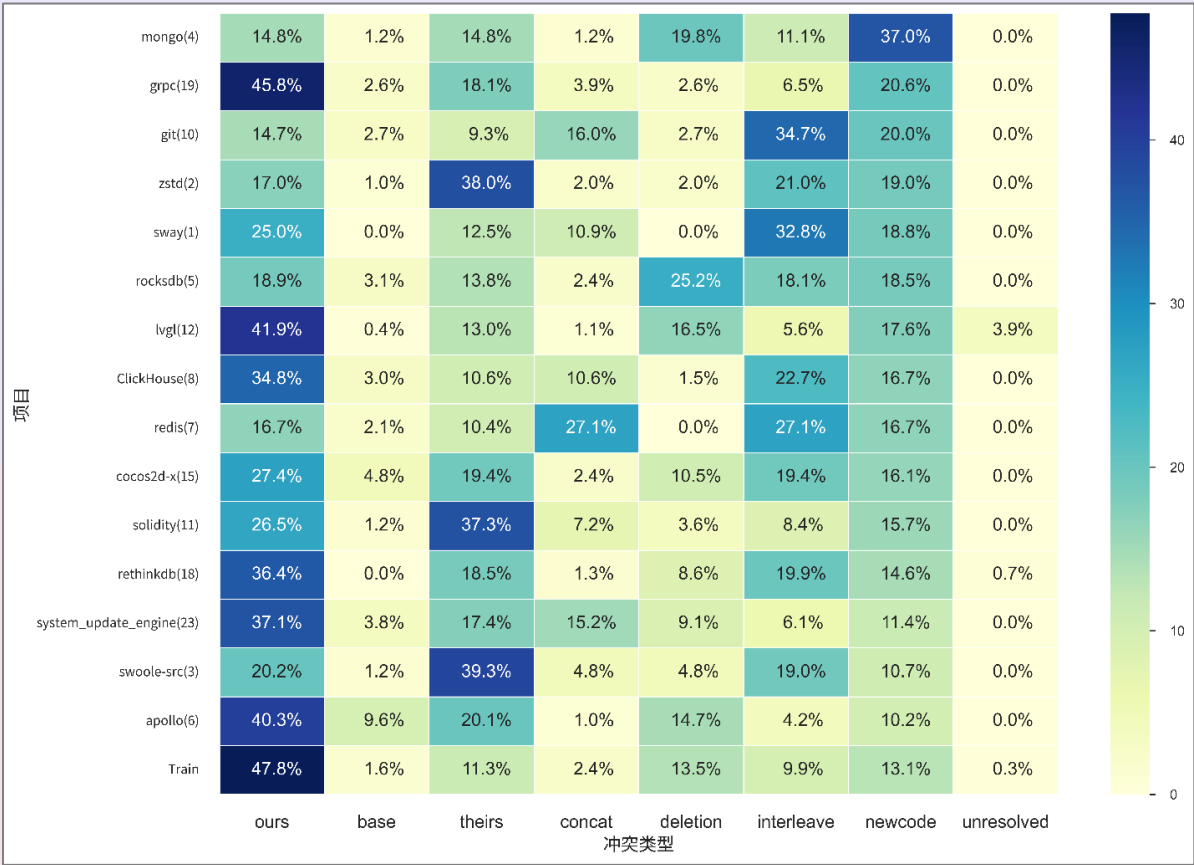
在600个被测合并冲突场景中，MergeSyn的中位运行时长为1842毫秒，MergeGen的中位运行时间为2152ms。虽然MergeSyn这类基于程序结构的方法采用了基于树的更复杂的冲突消解策略，但是超过90%的被测合并冲突场景都能在12秒内得到冲突消解的结果。

在实际持续集成流水线中，运行时间可接受。

- 将冲突消解策略划分为 ours, base, theirs, concat, deletion, interleave, newcode, unresolved 八类。
- 按 newcode 比例从大到小排序。
- 纵轴的序号表示MergeGen的实验结果中，Precision 从低到高的排序。

★ newcode 占比最高的15个项目中，有12个MergeGen 的精确率都很低，说明MergeGen对 newcode 的处理能力较差。

★ MergeGen 在解决由格式变动造成的冲突时其精确率仅为49.73%，而MergeSyn能达到97.80%。



基于预训练语言模型进行冲突消解的工作缺少对消解方案可靠性的评估，同时对冲突区块的特征学习不够，仅在进行自回归文本生成。

- 如果能评估模型生成冲突消解方案的可靠性?

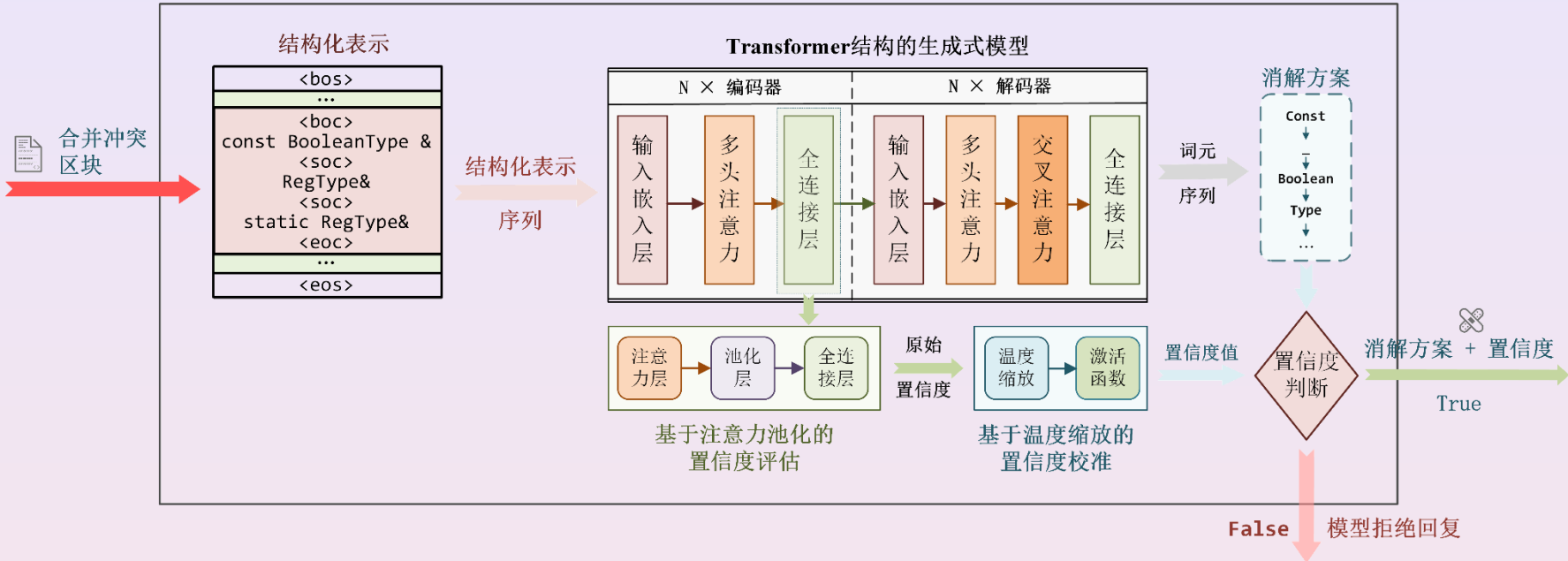


表 4.1 对比方法在测试数据集上的性能表现

对比方法	精确率 (Precision)	准确率 (Accuracy)
MergeSyn	56.31%	35.65%
MergeGen	39.18%	37.52%
MergeHesitate _{cali-0}	44.17%	42.71%
MergeHesitate _{cali-0.5}	56.49%	25.39%
MergeHesitate _{cali-0.6}	64.10%	16.22%
MergeHesitate _{cali-0.8}	89.32%	3.70%

★ 初步实验结果显示有效，能提升生成消解方案的可靠性，但是会大幅降低冲突的消解率。

★ 可能更好的方法是结合需求工程、测试生成等动态信息来做。

- Git的对象数据库和版本控制原理。
- 编程语言着色技术（多路径GLR文法）和代码补全技术（编程语言服务器协议）能够泛化部分程序静态分析任务。
- 合并冲突自动消解任务是一个健全性比完备性更重要的任务，确定性策略体现出了一定优势。
- 现有技术（基于程序结构的、学习驱动的、基于规则的）都难达到生成冲突消解方案的健全性，需要开发者人工的检查。
- 冲突消解方案的生成本质是一个需求工程问题（什么样的消解方案涵盖了两侧的修改），测试生成技术和动态程序分析可能有效。

感谢倾听

请各位老师同学批评指正

huahe@whu.edu.cn

